RESEARCH ARTICLE | MARCH 14 2025

Physics-informed Kolmogorov–Arnold networks: Investigating architectures and hyperparameter impacts for solving Navier–Stokes equations

Shuangwei Cui (崔双伟) ; Manshu Cao (曹曼殊); Yifeng Liao (廖益丰) ; Jianing Wu (吴嘉宁)

Physics of Fluids 37, 037159 (2025)
https://doi.org/10.1063/5.0257677

Physics of Fluids ARTICLE pubs.aip.org/aip/pof

Physics-informed Kolmogorov–Arnold networks: Investigating architectures and hyperparameter impacts for solving Navier–Stokes equations

Cite as: Phys. Fluids **37**, 037159 (2025); doi: 10.1063/5.0257677
Submitted: 11 January 2025 · Accepted: 20 February 2025 ·
Published Online: 14 March 2025

Shuangwei Cui (崔双伟),[1,a)] Manshu Cao (曹曼殊),[2,b)] Yifeng Liao (廖益丰),[1] and Jianing Wu (吴嘉宁)[1,3,c)]

AFFILIATIONS

[1]School of Aeronautics and Astronautics, Shenzhen Campus of Sun Yat-sen University, Shenzhen 518107, China
[2]School of Computer Science and Engineering, Guangzhou Campus of Sun Yat-sen University, Guangzhou 510006, China
[3]School of Advanced Manufacturing, Sun Yat-sen University, Shenzhen 518107, China

[a)]Electronic mail: cuishw@mail2.sysu.edu.cn
[b)]Electronic mail: caomsh@mail2.sysu.edu.cn
[c)]Author to whom correspondence should be addressed: wujn27@mail.sysu.edu.cn

ABSTRACT

In recent years, physics-informed neural networks have demonstrated remarkable potential in solving partial differential equations (PDEs). Typically constructed using multilayer perceptrons (MLPs), these networks integrate physical laws into their training process, enabling solutions for both forward and inverse problems. Recently, Kolmogorov–Arnold Networks (KANs) have emerged as a promising alternative to MLPs due to their superior interpretability and accuracy in small-scale tasks. In this study, we propose a Physics-Informed Kolmogorov–Arnold Network (PI-KAN) model to solve forward problems of the Navier–Stokes equations, a fundamental system in fluid mechanics known for its nonlinearity and complexity. We systematically investigate the effects of different network architectures, hyperparameters, and collocation point distributions on the accuracy and convergence of PI-KANs. We also conducted a comparative study between PI-KANs and MLP-based PINNs to contrast the characteristics of both neural networks in solving the Navier–Stokes equations. Specifically, we analyze the information bottleneck phenomenon in multi-output KANs and propose methods to address it by modifying hidden-layer configurations. Furthermore, we explore the impact of random seed initialization on training outcomes and evaluate the efficacy of a pruning-based approach for network optimization. Our results demonstrate that PI-KANs achieve high prediction accuracy for Navier–Stokes equations with well-designed architectures, the mean squared error between the predicted velocity and the true velocity in our constructed network has reached the order of 10^{-5}. Notably, uniform hidden-layer configurations yield optimal performance, while the balance between PDE and boundary condition losses plays a crucial role in achieving robust solutions. This study provides valuable insights into the design and implementation of PI-KANs for solving complex nonlinear PDEs, paving the way for broader applications in computational fluid dynamics and related fields.

I. INTRODUCTION

Recent advancements in machine learning (ML) have revolutionized various scientific disciplines by enabling the modeling of complex systems that were previously intractable using traditional methods. Deep learning architectures such as convolutional neural networks (CNNs)[1] and recurrent neural networks (RNNs)[2] have demonstrated exceptional capability in capturing intricate spatiotemporal dependencies, which has proven invaluable in simulating dynamic physical processes. These methodologies have been widely applied in fields ranging from climate modeling[3] and astronomy[4] to materials science and fluid dynamics.[5–9] In fluid mechanics, the Navier–Stokes equations represent one of the most foundational systems for describing fluid behavior.[10] Despite their widespread use in engineering and scientific applications, the inherent nonlinearity and high dimensionality of these equations pose significant challenges to obtaining analytical solutions. While traditional numerical methods, such as finite difference,[11] finite element,[12] and spectral techniques,[13] have been developed to approximate these solutions, they often require significant computational resources and struggle with highly nonlinear or ill-posed problems. Recent years have witnessed the emergence of machine-learning-based approaches to tackle these limitations. By leveraging data from

simulations or experiments, ML models can approximate solutions to partial differential equations (PDEs) at reduced computational costs. Among these, physics-informed neural networks (PINNs)[14] have gained considerable attention due to their ability to embed physical laws directly into the model architecture. Unlike purely data-driven approaches, PINNs integrate governing equations as constraints during training, enabling them to generalize better, especially in data-sparse scenarios. The concept of PINNs was first introduced by Raissi *et al.*,[14] who demonstrated their effectiveness in solving forward and inverse problems associated with various PDEs. These early studies highlighted PINNs' potential in addressing fluid mechanics problems, such as vortex-induced vibrations[14] and sparse data reconstruction. For instance, Jin *et al.*[15] utilized PINNs to simulate turbulence, achieving results comparable to direct numerical simulations (DNS). Cai *et al.*[16] extended PINNs to infer velocity and pressure fields from sparse temperature data, demonstrating their applicability to complex flow-field reconstruction. Further advancements have focused on enhancing the robustness and accuracy of PINNs. Eivazi and Vinuesa[17] proposed using PINNs for super-resolution flow-field reconstructions, demonstrating success in generating high-resolution outputs from noisy, sparse input data. Karniadakis *et al.*[18] provided a comprehensive overview of integrating physics into machine learning models, offering insights into PINN-based approaches for super-resolution, inverse modeling, and denoising applications. In parallel, Kolmogorov–Arnold Networks (KANs)[19] have emerged as a novel alternative to traditional PINN architectures, inspired by the Kolmogorov–Arnold representation theorem.[20–22] Liu *et al.*[19] explored the use of KANs for small-scale problems, highlighting their superior interpretability and accuracy in approximating complex functions. Unlike standard neural networks, KANs employ learnable activation functions, allowing greater flexibility in representing nonlinear relationships. These features position KANs as promising candidates for physics-informed modeling in fluid mechanics, offering a fresh perspective on solving nonlinear PDEs such as the Navier–Stokes equations.

The Navier–Stokes (NS) equations, as the fundamental equations of fluid mechanics, exhibit strong nonlinear characteristics. These inherent properties often lead to challenges when solving them using physics-informed neural networks (PINNs), such as difficulties in network training and low prediction accuracy of the trained model. Physics-Informed Kolmogorov–Arnold Networks (PI-KANs), a novel type of neural network for solving PDEs based on PINNs, remain relatively underexplored in their application to NS equations. Some researchers have investigated the use of KANs for solving NS equations, however, there has been a lack of systematic studies on the impact of network architectures, hyperparameters, and other factors on predictive performance. Liu *et al.*[19] conducted some analyses on how network parameters and architectures affect predictive accuracy, but their work focused exclusively on single-output networks. In contrast, our research employs a multi-output network with three outputs (u, v, and p), which may influence the effectiveness of existing methods. This underscores the necessity of a more systematic and comprehensive investigation into how network architectures and hyperparameters impact the performance of PI-KANs in solving the NS equations. Here, we chose the two-dimensional Taylor–Green vortex problem as our case study. First, the Taylor–Green vortex provides an analytical solution under specific conditions of the NS equations, fully reflecting their strong nonlinear characteristics. As an exact solution, it allows for precise evaluation of neural network prediction errors, which is crucial because solutions obtained through other numerical methods inherently include some level of approximation error. Second, the problem is inherently multi-output, with three outputs corresponding to u, v, and p, making it an ideal test case to assess the performance of multi-output PI-KANs.

II. METHODOLOGY

A. Multilayer perceptrons (MLPs)

Multilayer perceptrons (MLPs),[23–25] also known as fully connected neural networks, form the foundation of many modern deep learning architectures. They are universal function approximators capable of learning complex mappings between inputs and outputs through a hierarchical structure of neurons and nonlinear transformations. MLPs consist of an input layer, one or more hidden layers, and an output layer.[23,24] Each layer is composed of neurons, where every neuron in one layer is connected to every neuron in the adjacent layers, creating a fully connected topology.

The operation of an MLP begins with the input space $\mathscr{X}$, where input features are mapped to an output space $\mathscr{Y}$ through successive layers. For a given training dataset comprising input-output pairs $(\mathbf{x}, \mathbf{y})$, the goal of the MLP is to approximate the mapping $f : \mathscr{X} \to \mathscr{Y}$. This is achieved by optimizing the network parameters weights $\mathbf{W}$ and biases $\mathbf{b}$ —to minimize a predefined loss function[26] that measures the discrepancy between the predicted and true outputs.

In each layer, the input is transformed by a linear operation (weighted sum of inputs plus a bias), followed by a nonlinear activation function, such as the sigmoid,[27] ReLU,[28] or hyperbolic tangent functions.[29] Mathematically, the transformation at layer l can be expressed as

$$\mathbf{z}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\mathbf{z}^{(l-1)}$ represents the output from the previous layer, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias vector for layer l, and σ denotes the activation function. The output of the final layer provides the network's prediction.

One of the key strengths of MLPs lies in their ability to model arbitrary nonlinear functions, making them versatile for a wide range of applications. By stacking multiple layers, MLPs can represent intricate relationships between input and output spaces, which is particularly useful for high-dimensional and complex datasets. These networks have been successfully applied in domains such as image recognition,[30] natural language processing,[31] and numerical simulations.

However, MLPs face limitations, as training them often requires large datasets and significant computational resources, particularly when dealing with deep architectures. Additionally, the fully connected nature of MLPs can result in a high number of parameters, leading to overfitting[32] in cases where data are scarce. Despite these challenges, MLPs remain a cornerstone of neural network design and continue to inspire advancements in machine learning architectures.

In the context of physics-informed neural networks (PINNs), MLPs play a pivotal role in approximating solutions to partial differential equations (PDEs). The flexibility of MLPs allows them to encode complex spatial and temporal relationships inherent in physical systems. However, purely data-driven MLPs often fail to adhere to the underlying physics, necessitating the integration of governing equations into the training process, as seen in PINNs.

While MLPs have proven effective in many scenarios, their fixed activation functions and uniform architecture can limit their interpretability and efficiency. These limitations have motivated the development of alternative architectures, such as KANs,[19] which offer greater flexibility and interpretability, particularly for applications requiring strict adherence to physical laws.

B. Physics-informed neural networks

Physics-informed neural networks (PINNs)[14] represent a paradigm shift in solving partial differential equations (PDEs) by integrating physical laws directly into the training of neural networks. Unlike conventional neural networks, which rely solely on data to learn the underlying mappings between inputs and outputs, PINNs incorporate the governing equations of a physical system as soft constraints during training. This approach not only reduces the reliance on large datasets but also ensures that the learned solution adheres to the principles of the underlying physics.

In a PINN framework, the target solution is modeled using a neural network $\hat{u}(\mathbf{x};\theta)$, where $\mathbf{x}$ represents the input variables (e.g., spatial and temporal coordinates), and θ denotes the network parameters. Through the process of automatic differentiation,[33] the derivatives of the network's output with respect to the input variables can be computed, enabling the enforcement of PDE constraints. For example, if the PDE is expressed as

$$\mathcal{N}[\hat{u}(\mathbf{x})] = f(\mathbf{x}),$$

where $\mathcal{N}$ is a differential operator and $f(\mathbf{x})$ is a known source term; the residual of the equation $r(\mathbf{x}) = \mathcal{N}[\hat{u}(\mathbf{x})] - f(\mathbf{x})$ can be directly incorporated into the training objective.

The total loss function in PINNs typically combines multiple components, such as the residual loss for the PDE and additional losses for boundary or initial conditions. This can be expressed as

$$\mathcal{L} = \mathcal{L}_{\mathrm{PDE}} + \lambda_b \mathcal{L}_{\mathrm{BC}},$$

where $\mathcal{L}_{\mathrm{PDE}}$ measures the discrepancy between the neural network output and the PDE residual, $\mathcal{L}_{\mathrm{BC}}$ accounts for errors in satisfying the boundary or initial conditions, and λ_b is a weighting parameter to balance the contributions of different loss terms.

C. Taylor–Green vortex

The Navier–Stokes (N–S) equations are fundamental in fluid mechanics, providing a mathematical framework to describe the motion of viscous fluids.[34] These equations have been extensively validated in engineering and scientific applications, offering a reliable basis for studying fluid dynamics. However, their nonlinear nature poses significant challenges, often making it impossible to obtain analytical solutions for general cases. While numerical methods such as finite element and spectral techniques have been widely employed to approximate solutions, exact solutions exist only in a limited number of specific scenarios.[35,36] One such case is the Taylor–Green vortex,[37] a classic example of an exact solution to the incompressible Navier–Stokes equations.

The Taylor–Green vortex describes a decaying vortex in an unsteady flow, characterized by an exact closed-form solution in Cartesian coordinates. For a two-dimensional incompressible fluid with no body forces, the governing Navier–Stokes equations[38] are expressed as

$$\begin{cases} \dfrac{\partial u}{\partial x} + \dfrac{\partial v}{\partial y} = 0, \\ \dfrac{\partial u}{\partial t} + u\dfrac{\partial u}{\partial x} + v\dfrac{\partial u}{\partial y} = -\dfrac{\partial p}{\partial x} + \nu\left(\dfrac{\partial^2 u}{\partial x^2} + \dfrac{\partial^2 u}{\partial y^2}\right), \\ \dfrac{\partial v}{\partial t} + u\dfrac{\partial v}{\partial x} + v\dfrac{\partial v}{\partial y} = -\dfrac{\partial p}{\partial y} + \nu\left(\dfrac{\partial^2 v}{\partial x^2} + \dfrac{\partial^2 v}{\partial y^2}\right), \end{cases} \tag{1}$$

where u and v are the velocity components in the x and y directions, respectively, p is the pressure, and ν represents the kinematic viscosity. The first equation corresponds to the continuity equation, ensuring mass conservation, while the other two represent the momentum equations.

For the Taylor–Green vortex,[39] the velocity components are defined as

$$\begin{cases} u(x,y,t) = \sin(x)\cos(y)e^{-2\nu t}, \\ v(x,y,t) = -\cos(x)\sin(y)e^{-2\nu t}, \end{cases} \tag{2}$$

where t is time and ν controls the rate of viscous decay. These solutions satisfy both the continuity and momentum equations, providing an exact representation of the flow field. In this study, we focus on a two-dimensional implementation of the Taylor–Green vortex for validation purposes. The computational domain is defined as $[0, 2\pi] \times [0, 2\pi]$, with x and y representing the Cartesian coordinates. For simplicity, the fluid's temporal behavior is fixed at $t = 0$, yielding initial velocity components[39]

$$u(x,y) = \sin(x)\cos(y), \quad v(x,y) = -\cos(x)\sin(y). \tag{3}$$

These analytical solutions serve as benchmarks for evaluating the performance of the neural network models. By comparing the predicted velocity against the exact solutions, we can assess the accuracy and robustness of the KANs[19] approaches.

D. Kolmogorov–Arnold networks (KANs)

MLPs have long been a foundational architecture in modern deep learning, underpinning numerous applications and serving as the primary model for approximating complex functions. However, KANs[19] have recently emerged as a compelling alternative, offering enhanced flexibility and interpretability. Inspired by the Kolmogorov-Arnold representation theorem, KANs leverage a unique design to model multivariate functions with greater efficiency and accuracy.

The Kolmogorov–Arnold representation theorem states that any continuous multivariate function f defined on a bounded domain can be decomposed into a finite number of continuous univariate functions and additive operations. The network constructed based on the Kolmogorov-Arnold representation theorem can be expressed as:

$$f(\mathbf{x}) \approx \sum_{j=1}^{q} \phi_j \left(\sum_{i=1}^{n} g_{ij}(x_i) \right), \tag{4}$$

where q is the number of layers, n is the number of input dimensions, g_{ij} are univariate transformation functions, and ϕ_j are nonlinear activation functions. This formulation sets KANs apart from traditional MLPs by replacing the fixed activation functions with learnable functions at each edge, enabling more adaptive and expressive representations.

In their original implementation, Liu *et al.*[19] proposed defining the activation function $g_{ij}(x_i)$ as a combination of a base function and a spline function

$$g_{ij}(x_i) = \beta_0 b(x_i) + \sum_{k=1}^{m} \beta_k B_k(x_i), \tag{5}$$

where $b(x_i)$ is the base function (e.g., SiLU or ReLU), $B_k(x_i)$ are B-spline basis functions, and β_k are learnable coefficients. The flexibility of this representation enables KANs to efficiently capture intricate nonlinearities.

E. PI-KANs for NS equations

Physics-Informed Kolmogorov–Arnold Networks (PI-KANs) extend the capabilities of KANs by embedding physical laws, such as the NS equations, directly into the network's training process. This hybrid approach combines the interpretability and flexibility of KANs with the rigorous enforcement of governing physical principles, making it a powerful framework for solving complex fluid dynamics problems.

Figure 1 illustrates a schematic of the PI-KAN model applied to the NS equations. In a two-dimensional flow scenario, the PI-KAN model takes the spatial coordinates x and y as inputs and predicts the flow field variables, including the velocity components u and v, and the pressure p. Automatic differentiation is used to compute the derivatives of the network outputs, allowing the NS equations, comprising the continuity and momentum equations, to be formulated as residuals within the network's loss function. The PI-KAN model is designed to satisfy both the NS equations and the prescribed boundary conditions. The total loss function is defined as

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{PDE}} + \mathscr{L}_{\text{BC}}, \tag{6}$$

where $\mathscr{L}_{\text{PDE}}$ quantifies the residuals of the NS equations across the computational domain, and $\mathscr{L}_{\text{BC}}$ measures the error in satisfying boundary conditions. Here is the PDE residual loss ($\mathscr{L}_{\text{PDE}}$)

$$\mathscr{L}_{\text{PDE}} = \frac{1}{N_{\text{col}}} \sum_{i=1}^{N_{\text{col}}} \left(\mathscr{R}_1(\mathbf{x}_i)^2 + \mathscr{R}_2(\mathbf{x}_i)^2 + \mathscr{R}_3(\mathbf{x}_i)^2 \right) \tag{7}$$

where $\mathscr{R}_1, \mathscr{R}_2, \mathscr{R}_3$ are the residuals of the continuity and momentum equations at N_{col} collocation points within the domain. Here is the boundary condition loss ($\mathscr{L}_{\text{BC}}$)

$$\mathscr{L}_{\text{BC}} = \frac{1}{N_{\text{BC}}} \sum_{j=1}^{N_{\text{BC}}} \|\mathbf{u}_j - \mathbf{u}_{j,\text{true}}\|^2 \tag{8}$$

where $\mathbf{u}_j$ represents the predicted boundary values, and $\mathbf{u}_{j,\text{true}}$ denotes the known boundary conditions at N_{BC} boundary points.

In this framework, the collocation points within the domain enforce the NS equations by evaluating their residuals, while the boundary points ensure compliance with physical boundary conditions. The derivatives of u, v and p required for the residual computations are obtained via automatic differentiation, which allows seamless backpropagation of the loss gradients during training. The activation functions in PI-KANs are represented as learnable univariate splines, enhancing the network's adaptability to the underlying physics of the problem. Moreover, weighting factors can be introduced to balance the contributions of the PDE and boundary losses, accelerating convergence and improving accuracy.

III. RESULTS

A. A relatively larger KAN trained with different seeds

If the explicit solution to the PDE is known, we can manually design the architecture of the KANs, leveraging the interpretability advantages of KANs. In contrast, this is often challenging for MLPs. However, in most cases, the exact solution to the PDE is unknown. To address such scenarios, Liu *et al.*[19] proposed a pruning method. This method first constructs a large KANs and trains it using sparsification techniques on a dataset until machine precision is achieved. Then, each neuron is assigned a score, and neurons with scores below a predefined threshold are removed, resulting in a smaller network. The pruned network is subsequently retrained on the dataset to achieve machine precision. Ultimately, the trained KANs is capable of describing the target PDE. A neuron is considered important if both the incoming and outgoing scores exceed a threshold hyperparameter $\theta = 10^{-2}$. All neurons deemed unimportant are pruned from the network.

We trained a relatively large KANs with five layers, structured as [2,5,5,5,5,3], consisting of one input layer, four hidden layers, and one

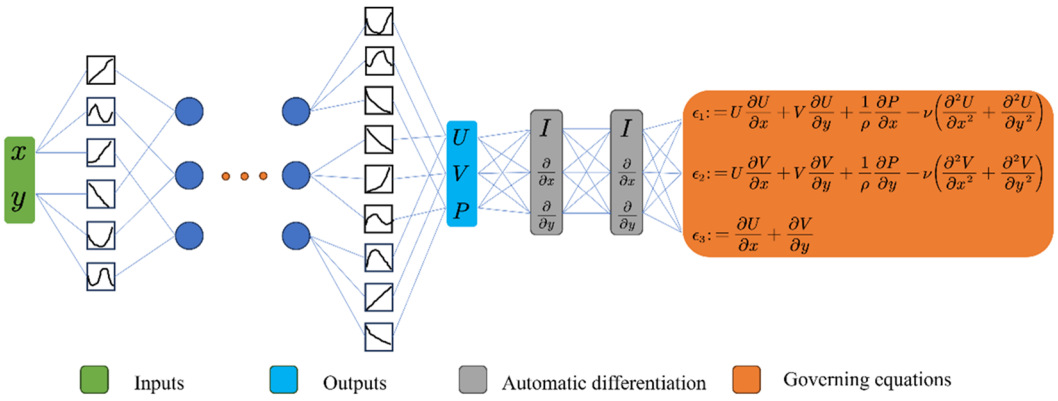

FIG. 1. A schematic diagram of the PI-KAN model for solving the Navier–Stokes equations.

output layer. The input layer has two inputs representing the spatial coordinates x and y, while the output layer contains three outputs corresponding to the velocity components u and v, and the pressure p. Along the boundaries of the computational domain, we uniformly placed 84 points where the physical quantities (i.e., u and v) serve as boundary conditions, calculated using the analytical solution. These boundary conditions constrain the neural network and guide it to optimize toward the true values at the boundaries.

In addition, we uniformly distributed 441 collocation points within the computational domain (21 points in the x direction and 21 points in the y direction). At these collocation points, only the spatial coordinates x and y are required. These points are used to enforce that the neural network satisfies the PDEs, specifically the NS equations. For now, we use this uniform distribution setup, and we will later investigate the effects of varying the number of collocation points on the network's predictive performance.

The neural network settings are as follows: the grid size is 3, and k is 3. The KANs employs splines along each edge to transform the data, where these splines are formed by linear combinations of basic B-splines. The grid parameter controls the number of these B-splines; a larger grid value results in more B-splines, enabling finer control over the composite splines. k represents the order of the splines. The noise scale is set to 0.3, and the base function used is SiLU. We employed the LBFGS[40] optimizer with a learning rate of 1. The weighting factor α is set to 0.1, where α determines the contribution of the pde loss to the total loss. Later, we will explore the impact of different values of α on the training process and the accuracy of the results.

Random seeds generally influence the initialization of neural networks and the random numbers used during training. However, to date, there has been little research on the impact of random seeds on the training of KANs and the accuracy of their predictions. In this section, we aim to investigate the effect of different random seeds on the training process under the current computational setup. We consider eight random seeds in total: 3, 16, 32, 64, 128, 256, 512, and 1024.

We trained the KANs for 200 steps with various random seeds, as illustrated in Fig. 2. Figures 2(a), 2(b), and 2(c) show the evolution of the bc loss, pde loss, and total loss over iterations for different seeds, respectively. Figure 2(d) presents a bar chart of the bc loss and pde loss across seeds at step 200.

From Fig. 2(a), it is evident that the bc loss converges rapidly across all seed cases, approaching near zero within the first 25 iterations for most scenarios. However, convergence rates differ slightly across seeds: the bc loss decreases fastest when seed = 16, whereas the slowest convergence occurs with seeds 512 and 1024. Nevertheless, bc loss decreases quickly under all conditions. This rapid convergence may be due to two factors. First, the bc loss is derived from the difference between the network's predicted physical values at the boundary and the actual values computed from a known analytical solution. This forms the supervised training component, making it easier for the network to optimize toward these known values. Additionally, the number of boundary points is approximately 1/5 of the number of collocation points within the domain, potentially contributing to an easier optimization. Second, because we set α to 0.1, the bc loss constitutes a larger proportion of the total loss, making the network more dependent on bc loss during optimization.

Figure 2(b) shows that, in all cases, pde loss decreases with iteration, though not as rapidly as bc loss. The rate of decrease is relatively fast for seeds 16, 32, and 64, with pde loss approaching near zero around 50 steps. However, for seed 1024, it takes until approximately 150 steps to reach a similar level, while for seed 512, the pde loss decreases the slowest, remaining above zero even at step 200. Figure 2(c) illustrates the total loss, calculated from bc loss and pde loss, which exhibits a trend similar to that of bc loss but with a decrease rate that falls between those of bc loss and pde loss.

Figure 2(d) provides a clearer view of the distribution of losses at the end of training (step 200) across different seed conditions. Here, bc loss has converged to a much lower value compared to pde loss. This suggests that while bc loss can reach very low values during training, pde loss may have a greater influence on the network's predictive performance. Notably, the total loss is minimized for seeds 16, 32, and 64, suggesting that these configurations may yield the best network predictions. Seeds 256 and 1024 show intermediate total loss values, while seeds 3 and 128 display higher losses. Particularly for seed 512, the total loss is significantly large, indicating that it is likely to yield the poorest predictive performance.

The detailed values of the three losses at the end of the computation (i.e., at steps = 200) for different seed values are listed in Table I. It can be observed that for the same seed, the bc loss is approximately two orders of magnitude smaller than the pde loss. Furthermore, when the seed is set to 512, the pde loss is the largest, being at least one order of magnitude greater than in other cases.

Figure 3 illustrates the predicted flow fields (velocity fields) by the neural network under different random seeds. The reference flow field, representing the exact flow solution, is derived from the analytical solution. These predicted flow fields can be grouped into three distinct categories. The first category includes seeds 16, 32, and 64. In these cases, the predicted flow fields closely match the exact flow field in both numerical range and velocity distribution characteristics. Specifically, the maximum velocity magnitude in each of these cases is 1.005, compared to the true flow field value of 1, resulting in a prediction error of $\frac{1.005-1}{1} \times 100\% = 0.5\%$. While the result for seed 64 shows a slight deviation, noticeable in the contours of the velocity isoclines, the velocity contours for seeds 16 and 32 appear regular and well-aligned with the true flow field.

The second category comprises seeds 3, 128, 256, and 1024. For this group, the prediction error in the maximum velocity is larger, but more critically, the flow field structure deviates significantly from the true flow field. The well-defined velocity contours in the true flow field appear blurred in these cases, and localized regions show larger discrepancies in the predicted velocity values. The third category consists of seed 512. By analyzing Fig. 2(d) and the specific values of pde loss in Table I, we observed that the pde loss in this case is significantly larger than in other cases, even by an order of magnitude. We hypothesized that the neural network's predicted flow field might be inaccurate under this condition, and this has now been confirmed. Although the error in the maximum velocity (0.5%) is minor, the flow field structure is far from accurate, diverging considerably from the true scenario. This indicates that, despite the numerical range being close to the true range, the structure of the flow field does not necessarily align.

From this analysis, we conclude that the seed significantly affects the convergence and prediction accuracy of a large KANs. When using PI-KANs for flow field predictions, it is advisable to test multiple random seeds to identify one that yields the most reliable computational results.

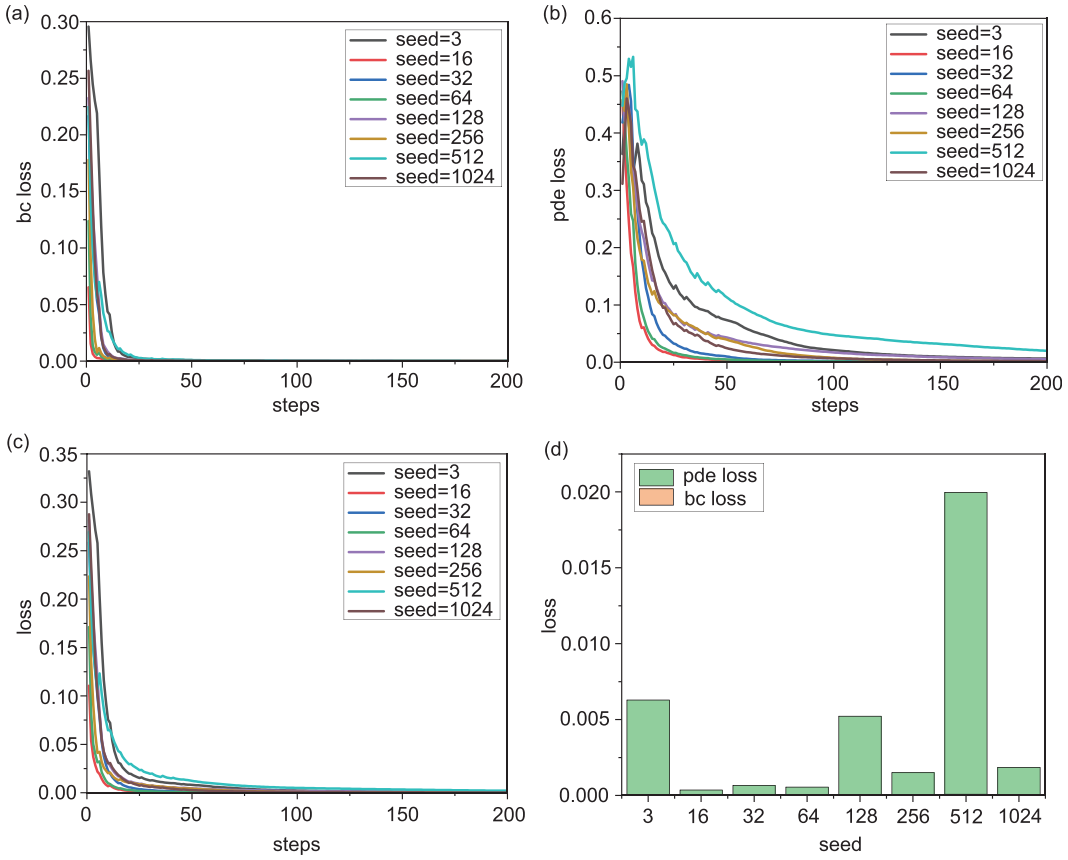

FIG. 2. Line plots showing the evolution of bc loss, pde loss, and total loss during the training process under different random seeds, along with a bar chart illustrating the distribution of bc loss and pde loss at step 200 for each random seed. The evolution of bc loss (a), pde loss (b), and loss (c) during training across 8 seed conditions, as well as the distribution bar chart of bc loss and pde loss at step = 200 for the 8 seed conditions (d).

TABLE I. A table showing the values of bc loss and pde loss at step 200 for different random seeds.

Losses	bc loss	pde loss
Seed = 3	4.19072×10^{-5}	0.006 25
Seed = 16	1.51268×10^{-6}	$3.526\,27 \times 10^{-4}$
Seed = 32	3.04143×10^{-6}	$6.540\,45 \times 10^{-4}$
Seed = 64	2.95969×10^{-6}	$5.477\,79 \times 10^{-4}$
Seed = 128	4.32751×10^{-5}	0.005 17
Seed = 256	8.6573×10^{-6}	0.001 49
Seed = 512	8.26556×10^{-5}	0.019 88
Seed = 1024	1.65759×10^{-5}	0.001 83

After training a large KANs and obtaining satisfactory prediction results by experimenting with different random seeds, we attempted to apply a pruning method to create a smaller network that accurately describes the flow field. However, a challenge arose: using a low contribution score threshold for automatic pruning did not yield effective results. We progressively increased the contribution score threshold, aiming to find a level that would reduce the network size. We conducted pruning with thresholds set at node_th=edge_th = 0.001, 0.01, 0.1, 0.2, and 0.3, where node_th is the contribution score threshold for nodes (activations for nodes scoring below this threshold are set to zero), and edge_th is the threshold for edges (edges scoring below this value are pruned). Nevertheless, the network structure remained unchanged until node_th=edge_th = 0.5, at which point the architecture was reduced from [2,5,5,5,5,3] to [2,5,5,5,3,3].

Despite this reduction, the pruned network architecture was still suboptimal. Therefore, we plan to manually design a network architecture based on insights from the form of the analytical solution to achieve an ideal prediction of the flow field. The original intent of pruning was to discard nodes or edges with minimal contributions (e.g., in the range of 10^{-2} or 10^{-3}), thus producing a more compact network. However, in this experiment, we had to increase the threshold to 0.5 before observing any reduction, meaning that the remaining nodes and edges made non-negligible contributions to the network output. Directly pruning these nodes and edges would therefore be unreasonable.

We hypothesize that the limited effectiveness of pruning in this computational scenario may be due to the network's multi-output nature (three outputs). Compared to single-output networks, multi-

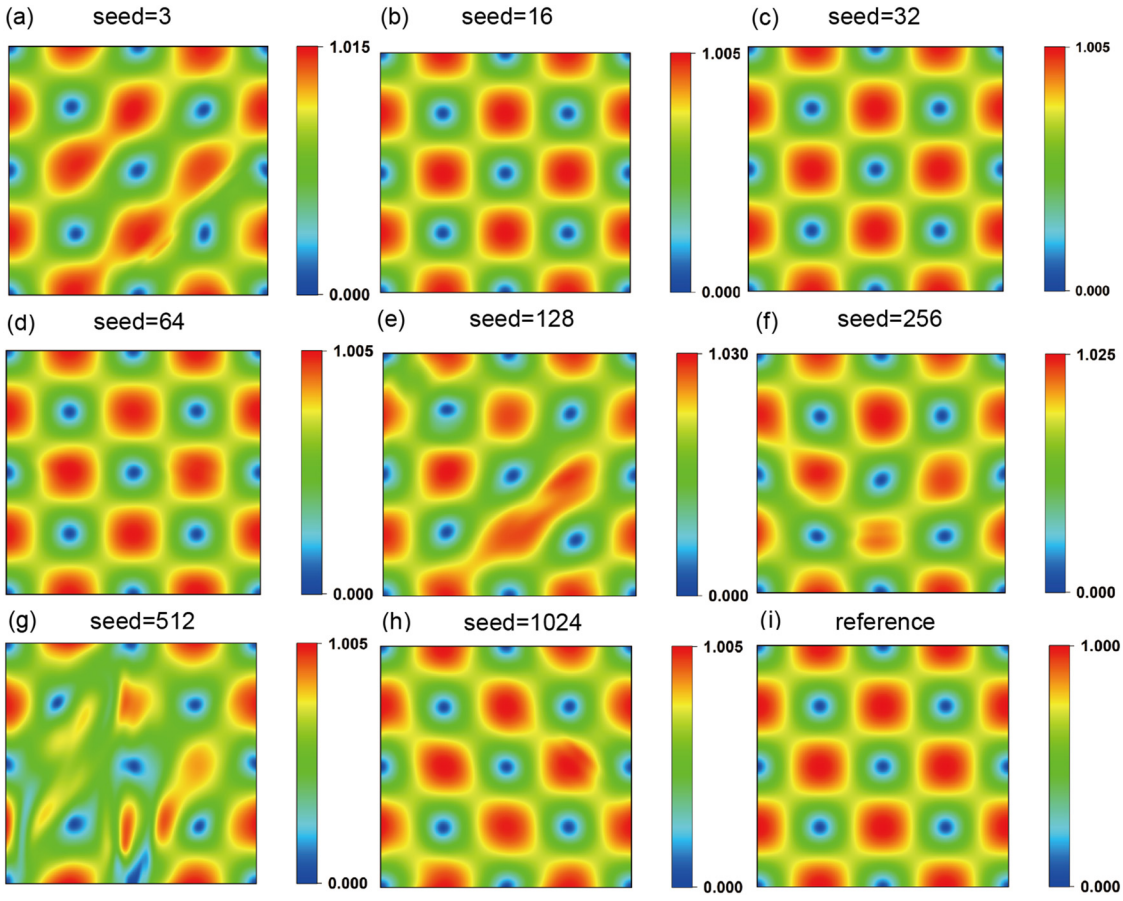

FIG. 3. The velocity fields predicted by the neural network under different random seeds, along with the velocity field obtained from the theoretical solution (reference). The predicted flow field results of the neural network for the cases of: (a) seed = 3, (b) seed = 16, (c) seed = 32, (d) seed = 64, (e) seed = 128, (f) seed = 256, (g) seed = 512, and (h) seed = 1024. (i) The velocity field obtained from the theoretical solution.

output networks may diminish the effectiveness of pruning, as the distribution of contributions across outputs likely reduces the overall sparsity.

We aim to use Mean Squared Error (MSE) to quantitatively assess the discrepancy between the neural network's predicted velocity field and the true velocity field. The MSE is defined as

$$\sum_{i=1}^{n} \| V^i_{exact} - V^i_{pred} \|_2^2,$$

where V represents the velocity magnitude, V^i_{exact} is the true velocity at the i-th collocation point, and V^i_{pred} is the predicted velocity at the same point. The notation $\| \cdot \|_2^2$ refers to the squared 2-norm of the matrix.

Before proceeding with this metric, we first explore whether the MSE computed over the collocation points used in training can adequately represent the MSE over the entire flow field domain. Since collocation points are directly involved in the network's training process, the neural network specifically adjusts its parameters to satisfy the NS equations at these points. Therefore, the MSE calculated over these points is expected to be minimal upon training completion. However, to fairly assess the network's accuracy, MSE should ideally be computed over a large set of randomly selected points throughout the flow field, requiring a broader sampling across the domain.

Unlike traditional numerical methods, such as the finite volume method (FVM), which discretizes PDE on a mesh and interpolates to compute quantities at various positions, neural networks represent a mesh-free approach. Some perspectives suggest that neural networks learn the high-dimensional manifold of the data. In this study, we hypothesize that the network learns the manifold structure of the NS equations. Once trained, with fixed parameters, the neural network interpolates on the learned manifold when predicting quantities at positions beyond the collocation points. Thus, even if the predictions are accurate at the collocation points, interpolation accuracy across the rest of the domain relies on the quality of the learned manifold.

The number of collocation points selected for training is subjective. Ideally, increasing the number of collocation points should improve prediction accuracy but at the cost of extended training time. Conversely, insufficient collocation points may lead to suboptimal prediction accuracy. The optimal number of collocation points should be such that the MSE on these points closely approximates the MSE across the entire flow field. By iteratively reducing the number of

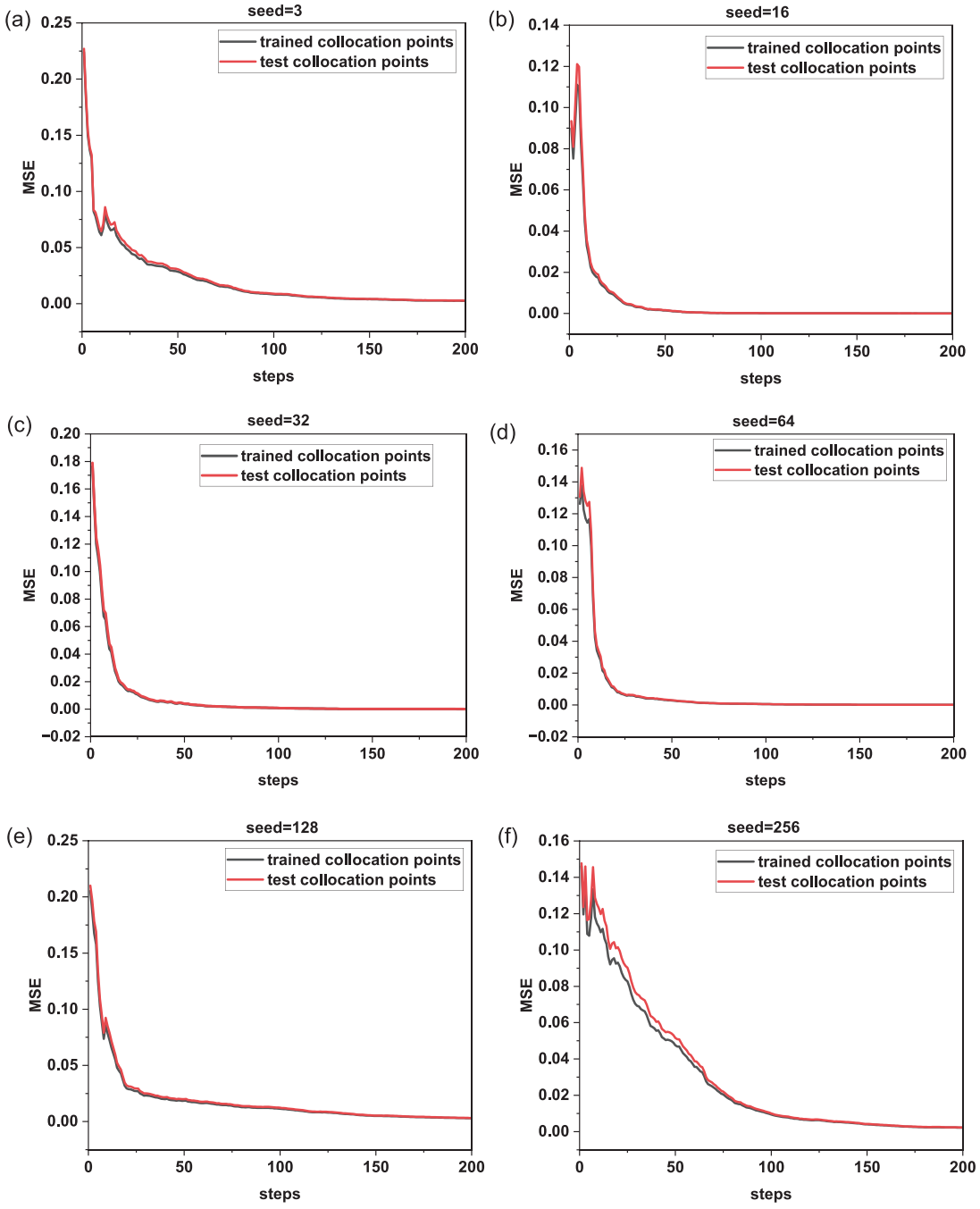

FIG. 4. The evolution of the MSE over training steps for different seeds on both trained and test collocation points. The MSE evolution curves over training iterations for the neural network on both training and test collocation points under the following seed conditions: (a) seed = 3, (b) seed = 16, (c) seed = 32, (d) seed = 64, (e) seed = 128, (f) seed = 256, (g) seed = 512, and (h) seed = 1024.

collocation points until the training MSE no longer approximates the true MSE, we can achieve a balance between network accuracy and training efficiency.

In this training, we used 21^2 collocation points, with 21 points evenly spaced in both the x and y directions, denoted as "trained collocation points." For testing, we sampled 500^2 points (500 evenly spaced points in both x and y directions), referred to as "test collocation points." We computed the MSE on both sets of points, and Fig. 4 shows the evolution of the MSE over training iterations for different seeds on both trained and test collocation points. Across all seeds, the

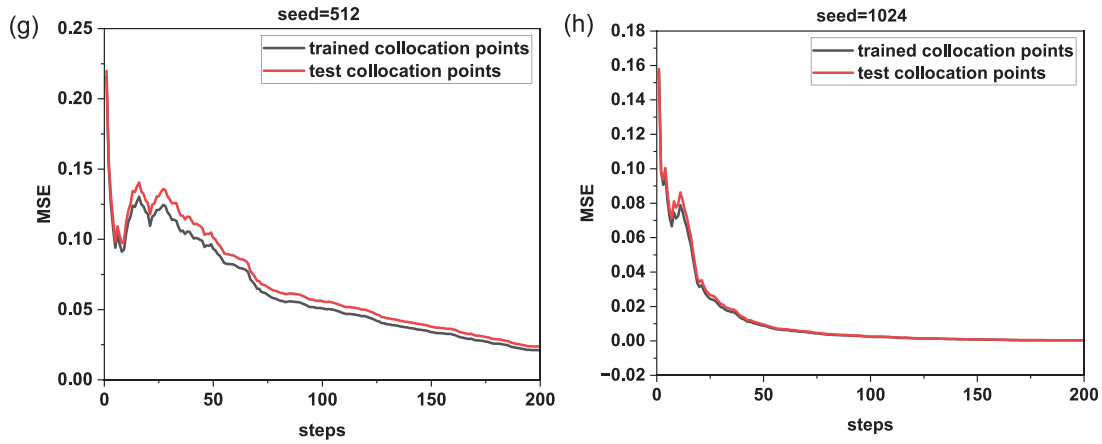

FIG. 4. (*Continued.*)

two MSE curves closely align, especially toward the end of training, indicating that the MSE on the trained collocation points can reliably approximate the MSE over the test points. Therefore, monitoring the MSE on the trained collocation points during training can serve as a direct indicator of the network's predictive accuracy.

Figure 5 shows the distribution of MSE at the end of training (steps = 200) for different random seeds, with MSE computed over the trained collocation points. As seen in the figure, the MSE is lowest for seeds 16, 32, and 64, approaching values close to zero. In contrast, the MSE increases by an order of magnitude for seed 1024, then increases further by another order of magnitude for seeds 3 and 128, and by yet another magnitude for seed 256. These MSE values reflect the prediction accuracy of the flow fields shown in Fig. 3. The results highlight that, even with identical computational settings, the choice of random seed can significantly impact both the accuracy and convergence rate of the neural network's predictions.

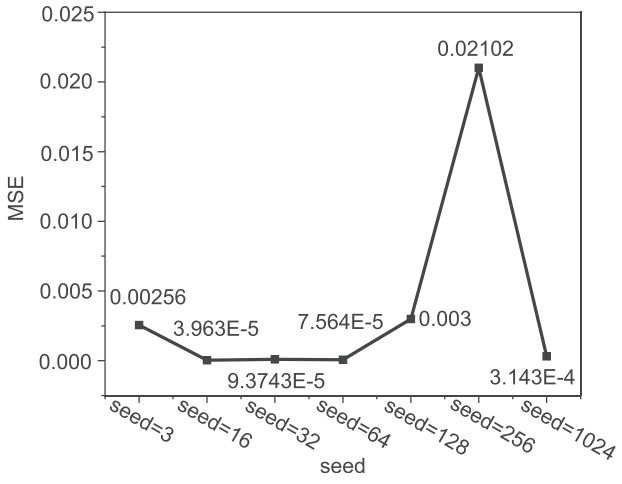

FIG. 5. The distribution of MSE at the end of training (steps = 200) for different random seeds, with MSE computed over the trained collocation points.

B. Information bottleneck in multi-output KANs

The neural network used in this study has two inputs and three outputs, in contrast to the single-output networks primarily studied by Liu *et al.*[19] For multi-output networks, neurons in the earlier layers need to carry sufficient information to represent multiple outputs. This demand for greater informational capacity in each neuron layer is significantly higher than in single-output networks. For a single-output network, we can generally construct a minimal architecture based on the output variable's form (if known), or at least one that closely approaches the minimal configuration; any remaining optimization can then be achieved through pruning. However, this direct approach to constructing the network architecture becomes challenging for multi-output networks. For instance, if we only output u, and its form is known as u=sin(x)cos(y), a [2,2,1] architecture is sufficient to represent this function. To investigate the information bottleneck in multi-output networks, we constructed a network with a [2,2,3] architecture and used it to train and predict flow fields.

We set the network's random seed to 16, used a grid with resolutions of 5, 10, and 20 at $\alpha = 0.1$, and, for the case where the grid was set to 20, also set α to 1.0. All other configurations matched those in Sec. III A. These four configurations were analyzed to assess the neural network's performance in predicting flow fields. Figure 6 presents the loss and MSE curves throughout the training process across these cases. In Fig. 6(a), where $\alpha = 1$ and grid resolution is 5, we observe that the MSE initially decreases to a minimum of about 0.04 by step 5, after which it rises and stabilizes, indicating overfitting. Other cases also exhibit some degree of overfitting. In general, increasing grid resolution tends to improve network performance, but the lowest MSE values across these cases were 0.04, 0.03, 0.07, and 0.1, none of which approach zero, with the lowest reaching only on the order of 10^{-2}. In fact, the flow field predictions under these configurations are poor. Increasing α to 1.0 to boost the contribution of pde loss in the optimization did not enhance accuracy; it actually decreased it. These observations suggest that the [2,2,3] network configuration suffers from an information bottleneck, as the two neurons in the intermediate layer are insufficient to carry the amount of information required to accurately represent three outputs.

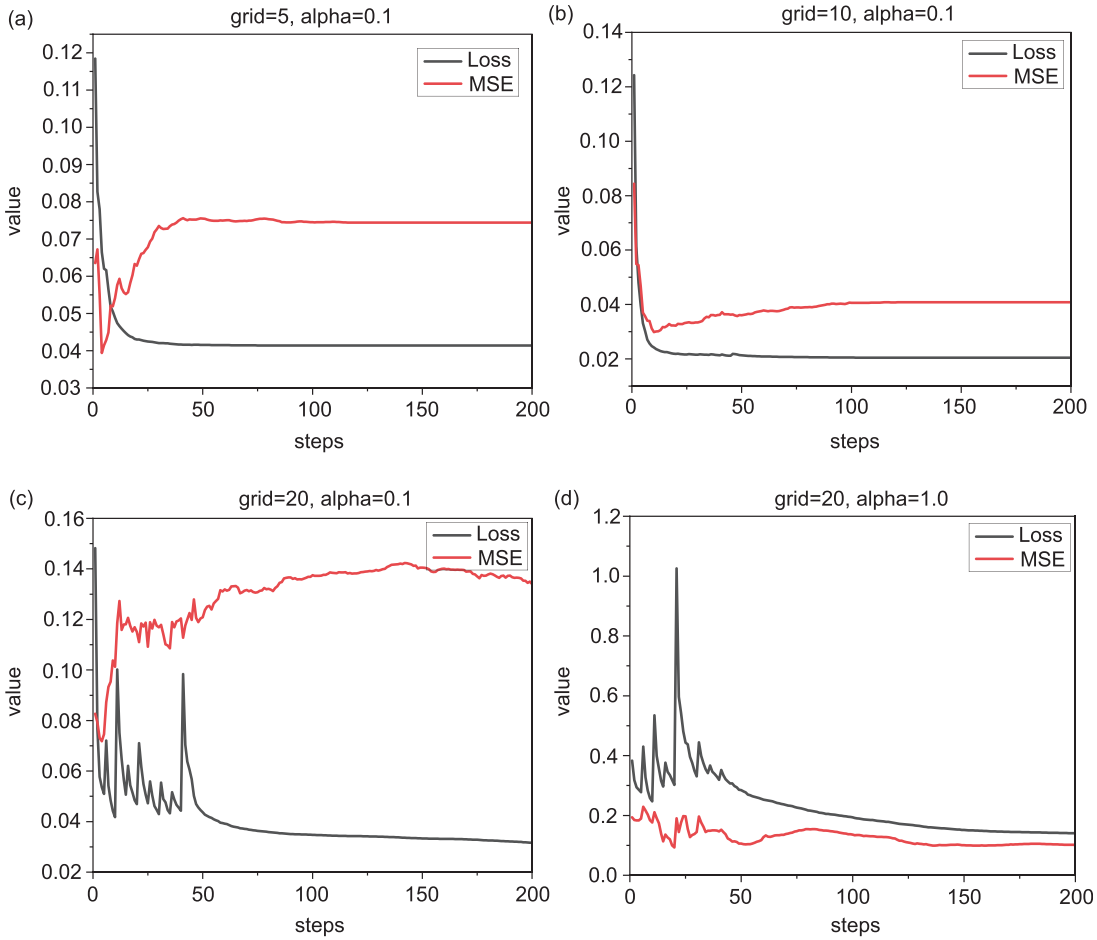

FIG. 6. The evolution curves of loss and MSE during the training process under the network architecture [2,2,3] for different grid and α values. The evolution curves of total loss and MSE over training iterations for the network with architecture [2,2,3] under the following conditions: (a) grid = 5, $\alpha = 0.1$, (b) grid = 10, $\alpha = 0.1$, (c) grid = 20, $\alpha = 0.1$, (d) grid = 20, $\alpha = 1.0$.

To address the information bottleneck observed in the [2,2,3] neural network configuration, we added one neuron to the intermediate layer, resulting in a [2,2,3] architecture. The network was trained under the conditions of seed = 16 with grid resolutions of 5, 10, and 20, and the training loss and MSE were recorded. As shown in Fig. 7, the addition of one neuron in the intermediate layer significantly improved the network's prediction accuracy.

In Figs. 7(a) and 7(b), the final MSE values decreased to 0.0047 and 0.00129, respectively, both on the order of 10^{-3}. In contrast, the MSE in Fig. 7(c) was 0.01779, comparable in magnitude to the [2,2,3] network, indicating suboptimal training performance. Notably, increasing the grid resolution in this case worsened the prediction accuracy, which contradicts our general experience. Additionally, the loss curve in this case exhibited significant oscillations during training, potentially due to the poor optimization landscape caused by the larger grid. Among the three configurations, the best results were achieved with grid = 10 and $\alpha = 0.1$, yielding a prediction accuracy that the [2,2,3] architecture failed to achieve.

Subsequently, we increased α to 1.0 while keeping grid = 10. As shown in Fig. 7(d), the final MSE was 0.07086, the worst among the four scenarios. Combined with the observations from Fig. 6, we hypothesize that increasing the relative weight of the pde loss (α) may negatively impact the network's training performance.

We further increased the number of neurons in the hidden layer to four, resulting in a [2,4,3] neural network configuration. This network was trained with seed = 3 and grid = 10 and was used to predict the velocity field, achieving significantly improved prediction accuracy. Figure 8 illustrates the predicted velocity fields for the three network architectures: Figs. 8(a) and 8(b) correspond to the cases with the smallest MSE for the [2,2,3] and [2,3,3] networks, respectively, while Fig. 8(d) shows the reference velocity field.

From the comparison in Fig. 8, it is evident that the prediction accuracy of the [2,2,3] network remains poor despite efforts to adjust various network parameters. This outcome is likely due to the information bottleneck effect caused by the insufficient number of neurons in the hidden layer. The [2,3,3] network exhibits improved performance,

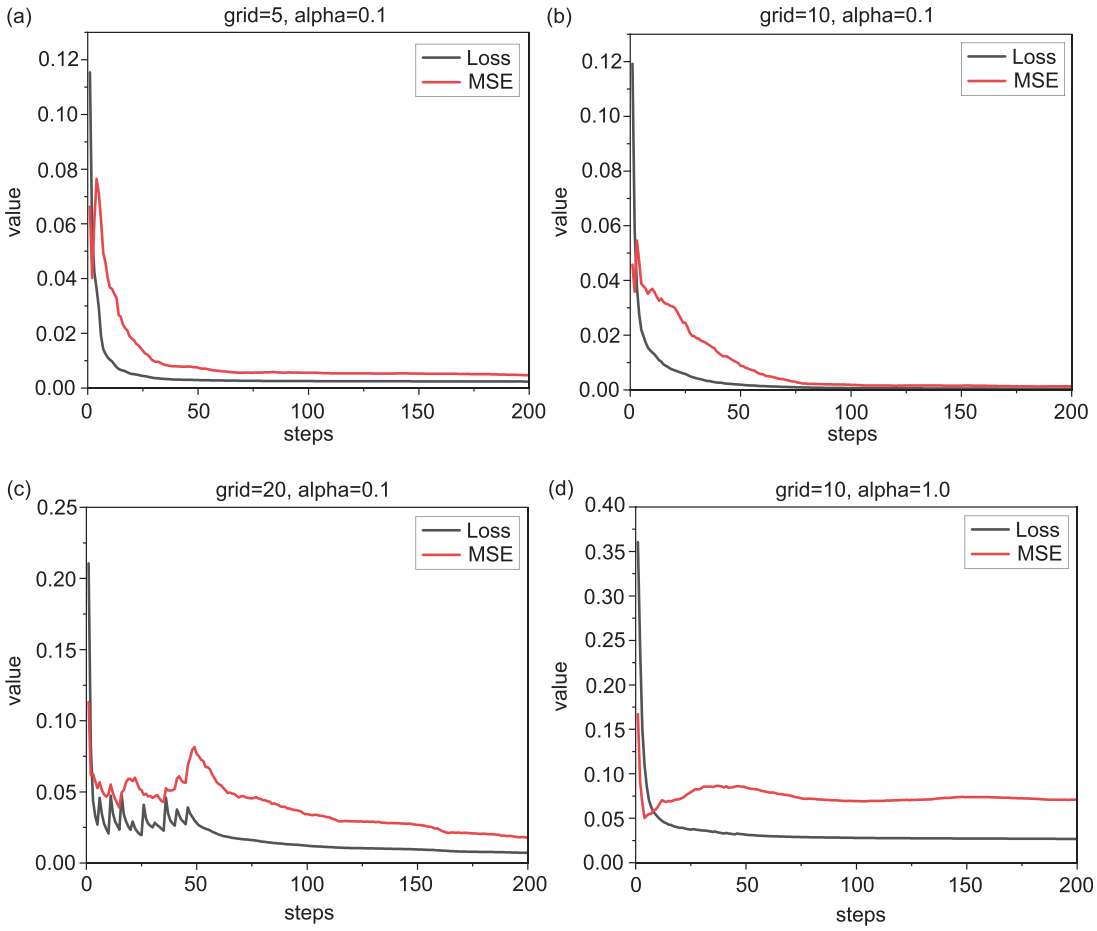

FIG. 7. The evolution curves of loss and MSE during the training process under the network architecture [2,3,3] for different grid and α values. The evolution curves of total loss and MSE over training iterations for the network with architecture [2,3,3] under the following conditions: (a) grid = 5, $\alpha = 0.1$, (b) grid = 10, $\alpha = 0.1$, (c) grid = 20, $\alpha = 0.1$, and (d) grid = 20, $\alpha = 1.0$.

but its prediction accuracy is still suboptimal in localized regions of the velocity field. In contrast, the [2,4,3] network achieves much better prediction results, demonstrating that increasing the number of neurons in the hidden layer effectively breaks the information bottleneck and allows the network to better represent the three output variables.

C. Influence of collocation point numbers on the trends of two types of MSE and neural network prediction accuracy

The number of collocation points used in training has a significant impact on the trends of MSE and the prediction accuracy of the neural network. We analyzed the behavior of two types of MSE: one computed at the training collocation points (trained MSE) and the other evaluated across a dense grid of test points representing the entire domain (test MSE). These two metrics help us understand how well the network generalizes beyond the collocation points.

We used a network architecture of [2,4,3] to investigate how the number of collocation points affects the neural network's prediction accuracy for the velocity field and the trends of two MSE metrics: trained MSE (calculated at the collocation points used during training) and test MSE (calculated at a dense grid representing the entire domain). In this study, we considered five cases with uniformly distributed collocation points, where the number of points (npi) was set to 5, 6, 7, 9, and 11. Figure 9 illustrates the evolution of both MSE metrics over iterations for different values of npi.

For npi = 5: The trained MSE was approximately twice as large as the test MSE, with both converging poorly; The predicted velocity field was highly inaccurate, which was unexpected. As the neural network directly uses the collocation points during training to minimize the loss, the trained MSE is typically expected to be smaller than the test MSE; This counterintuitive result suggests that an extremely sparse collocation point distribution might lead to overfitting or poor generalization, causing trained MSE to exceed test MSE significantly.

For npi = 6: Both MSE metrics showed substantial reductions compared to npi = 5; however, the test MSE was still about 30% higher than the trained MSE; Despite the improvement, the neural network's prediction accuracy remained low, failing to capture essential characteristics of the velocity field.

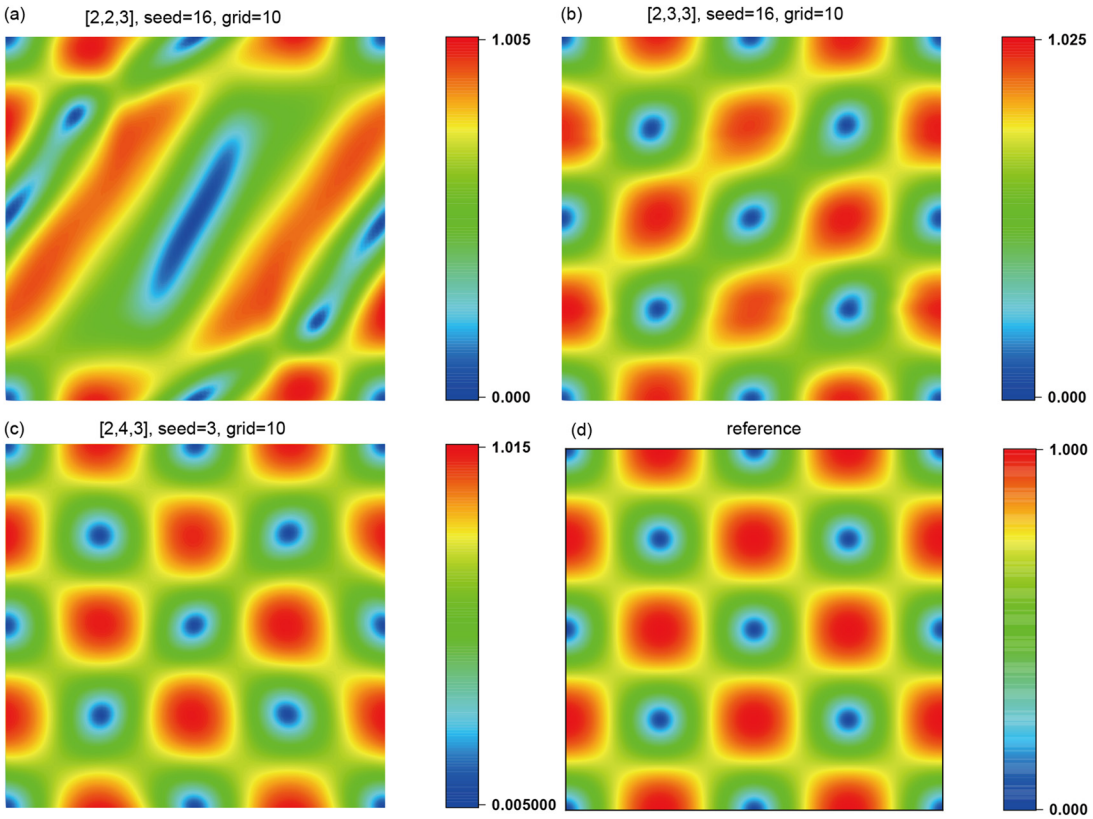

FIG. 8. The predicted velocity fields for three network architectures. (a) The velocity field corresponding to the network architecture [2,2,3] with seed = 16 and grid = 10; (b) the velocity field corresponding to the network architecture [2,3,3] with seed = 16 and grid = 10; (c) the velocity field corresponding to the network architecture [2,4,3] with seed = 3 and grid = 10; (d) the reference velocity field.

For npi = 7 and npi = 9: Further increases in the number of collocation points resulted in continued decreases in both MSE metrics; The trained MSE was slightly smaller than the test MSE, which aligns with expectations; However, the predicted velocity field was still highly inaccurate, with the network struggling to learn key features of the flow field.

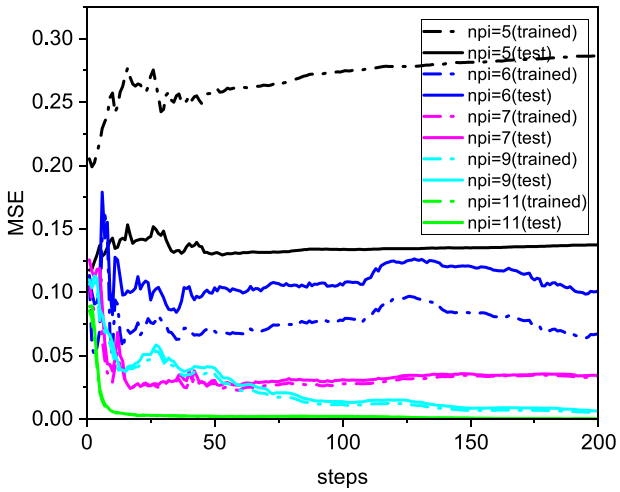

FIG. 9. The evolution of both MSE metrics over steps for different values of npi.

For npi = 11: Both MSE metrics dropped significantly, and their descent rate over iterations was notably faster than in previous cases; The curves for trained MSE and test MSE overlapped almost perfectly, indicating that trained MSE could reliably represent test MSE in this case; The predicted velocity field was highly accurate, capturing the flow field's features with excellent precision.

As the number of collocation points increases, the accuracy of the neural network in predicting the velocity field improves significantly, as reflected by the declining values of both MSE metrics. However, when the collocation points are too sparse (npi $\leq$ 9), the network struggles to learn the flow field effectively, leading to poor prediction accuracy. A sufficient number of collocation points (npi = 11 in this study) ensures that the trained MSE closely matches the test MSE, indicating reliable generalization and accurate velocity field prediction. This analysis highlights the importance of selecting an adequate number of collocation points to balance computational cost and prediction accuracy.

D. Training performance of KANs with different architectures under equal parameter constraints

In this section, we investigate the performance of KANs under the condition of equal parameter counts, comparing different

architectural configurations to determine the most effective structure for neural network training. The primary objective is to understand how variations in the number and distribution of neurons across layers influence the training dynamics and final performance of the network.

We constructed three different KANs architectures, each with an equal number of parameters, but varying neuron distributions across hidden layers. The three architectures are as follows:

1. N-shaped Architecture: [2,4,7,4,3] – The number of neurons in the hidden layers increases initially, then decreases symmetrically.
2. Uniform Architecture: [2,5,5,5,3] – Each hidden layer contains the same number of neurons, resulting in a uniform distribution.
3. U-shaped Architecture: [2,6,3,6,3] – The number of neurons decreases in the hidden layers and increases symmetrically in the outer layers.

These architecture were selected to explore the impact of different hidden layer configurations on the training efficiency and accuracy of KANs under the constraint of equal total parameters.

For training, we used the settings outlined in Sec. III A, applying the same training procedure to all three network architectures. The networks were trained for 100 iterations using a standard dataset, and their performance was evaluated based on the loss and MSE values. We focused on assessing the ability of each architecture to learn and generalize, particularly its capacity to approximate the target function accurately.

Figure 10(a) shows the loss evolution during training for the three network architecture. The loss curves for all three architectures followed a similar pattern. However, the U-shaped architecture ([2,6,3,6,3]) exhibited a slightly faster loss reduction during the initial 20 iterations compared to the N-shaped and Uniform architecture. Despite these initial differences, all architecture eventually converged to a final loss value, with the N-shaped architecture achieving the lowest convergence loss. The final loss values are: N-Shaped: 5.46×10^{-4}, Uniform: 2.46×10^{-4}, U-Shaped: 1.85×10^{-3}.

Figure 10(b) presents the MSE for each architecture during training. Both the N-shaped and Uniform architectures achieved relatively low MSE values, with the N-shaped architecture reaching 3.06×10^{-3} and the Uniform architecture reaching 6.85×10^{-4}. In contrast, the U-shaped architecture showed a much higher MSE of 0.1575, indicating significantly poorer prediction performance. These results suggest that, under equal parameter constraints, the Uniform and N-shaped architectures perform similarly well in approximating the desired function, while the U-shaped architecture struggles due to its suboptimal configuration.

The predictive accuracy of the networks was also evaluated by comparing the predicted velocity fields to the theoretical reference solution. The N-shaped and Uniform architectures generated relatively accurate velocity fields, while the U-shaped network produced significantly poorer results, consistent with its higher loss and MSE values.

Based on the training results, we conclude that the Uniform architecture offers the best performance under equal parameter constraints, closely followed by the N-shaped architecture. Both of these architectures achieved accurate predictions and converged efficiently during training. In contrast, the U-shaped architecture performed poorly, with significantly higher MSE and less accurate flow field predictions.

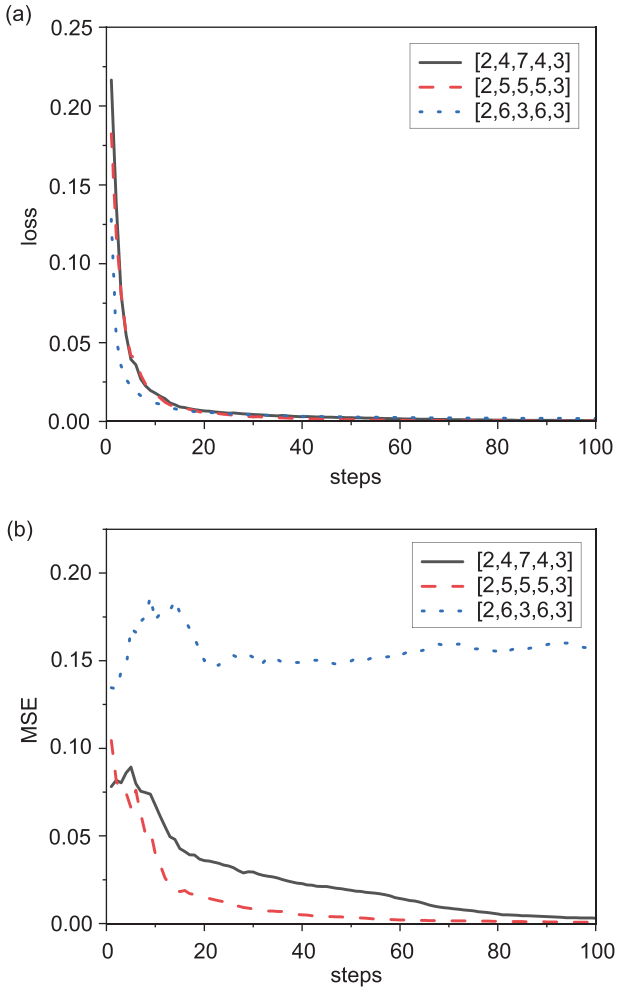

FIG. 10. The evolution of loss and MSE during the training process for three different network architectures. (a) The loss evolution during training for the three network architectures. (b) The MSE for each architecture during training.

E. Comparison with MLP

Kolmogorov–Arnold Networks (KANs), as a novel neural network architecture that can replace traditional MLPs, offer advantages in terms of higher accuracy and better interpretability when applied to certain scientific problems. In this section, we compare the performance of Physics-Informed Neural Networks (PINNs) constructed using MLP and KAN architectures for solving the Taylor–Green vortex problem, specifically evaluating the accuracy and performance of both network architectures.

We first trained a PI-KAN with the architecture [2,5,3] for the Taylor–Green vortex problem, using the following domain and network settings: The computational domain is the same as mentioned earlier, i.e., $[0, 2\pi] \times [0, 2\pi]$, with a fluid viscosity coefficient $\nu = 0.01$. The number of collocation points within the computational domain is 441, and 84 points are uniformly and randomly selected on each of the four boundaries, with Dirichlet boundary conditions applied. A third-order B-spline curve is used as the trainable activation function, the

grid is set to 5, and the total number of parameters is 105. The seed is set to 32, and the LBFGS optimizer is used with a learning rate of 1.0. After training for 100 steps, we observed that the total loss had nearly converged. We calculated the MSE for the predicted velocity components u and v, which were found to be 0.0021 and 0.0019, respectively, with a total training time of approximately 18 minutes. These results were used as a benchmark for comparing with MLP-based PINNs.

For the MLP-based approach, we constructed fully connected PINNs, which is a commonly used method. We built a three-layer MLP and a five-layer MLP, with the following configurations: The three-layer network has one input layer, one hidden layer with five neurons, and one output layer with three neurons. The five-layer network has intermediate hidden layers with 10, 20, and 30 neurons. We denote these networks as mlp_1, mlp_2, mlp_3, and mlp_4, corresponding to the shapes [2,5,3], [2,10,10,10,10,3], and [2,20,20,20,20,3], respectively. We used the tanh activation function and the Glorot normal weight initialization method, which is suitable for fully connected layers and automatically adjusts the initialization range based on input and output dimensions. We trained the models for 30,000 steps using the Adam optimizer, switching to the LBFGS optimizer for additional training steps. The number of collocation points in the domain and on the boundaries remained the same as in the PI-KAN training (441 and 84 points, respectively).

The training results for the four MLP models are as follows:

mlp_1: The architecture of [2,5,3] was constructed for direct comparison with the PI-KAN model. Although the shapes of the two networks are the same, the total number of parameters differs due to different parameter calculation methods. The total number of parameters for mlp_1 is 33, which is reasonable since KANs have more parameters than MLPs with the same architecture. During training, we monitored the training and testing losses, and found that the best prediction performance occurred at step 19 000, with a training time of approximately 6 minutes. The final MSE for the velocity components u and v were 0.1660 and 0.1402, respectively.
mlp_2: This model has 393 parameters. After training with the Adam optimizer for 30 000 steps, we switched to the LBFGS optimizer, training for 33 737 steps to achieve the best results. The final training time was approximately 14 minutes, with MSEs of 0.0954 and 0.0916 for u and v, respectively.
mlp_3: With a total of 1383 parameters, the model trained similarly to mlp_2, switching to LBFGS after 30 000 steps to achieve the best performance. The final MSEs for u and v were 0.0066 and 0.0089, respectively, with a training time of approximately 13 minutes.
mlp_4: With 2973 parameters, the model trained similarly to mlp_3 but required 34 680 steps for the best results. The final MSEs for u and v were 0.000036 and 0.000039, respectively, with a training time of approximately 18 minutes.

The training for all networks was performed on an Intel Core i7-11800H CPU. For the problem studied in this paper, PI-KANs required more time per iteration compared to MLP-based PINNs. However, PI-KANs typically achieved better accuracy with fewer training iterations. For example, after training PI-KANs for just 100 steps, the results were comparable to those obtained by training mlp_4 for more than 30 000 steps, with the total training time being roughly the same. Furthermore, we set the learning rate for PI-KANs to 1.0, whereas MLPs were trained with a learning rate of 0.001. Attempts to increase the learning rate for MLPs resulted in poor convergence due to the large number of network parameters, whereas PI-KANs, with fewer parameters, still converged quickly even with a larger learning rate.

Figure 11(a) shows the MSE curves for the velocity components u and v for the networks trained using the various models. It is clear from the figure that the MSE for the PI-KAN with the [2,5,3] architecture is much smaller, while the MSE for the same shape MLP model is approximately eight times larger. Even after switching to the LBFGS optimizer and training for additional steps, the MLP model does not achieve a significant improvement, which may be due to information bottleneck issues. As we increase the hidden layers and neurons for

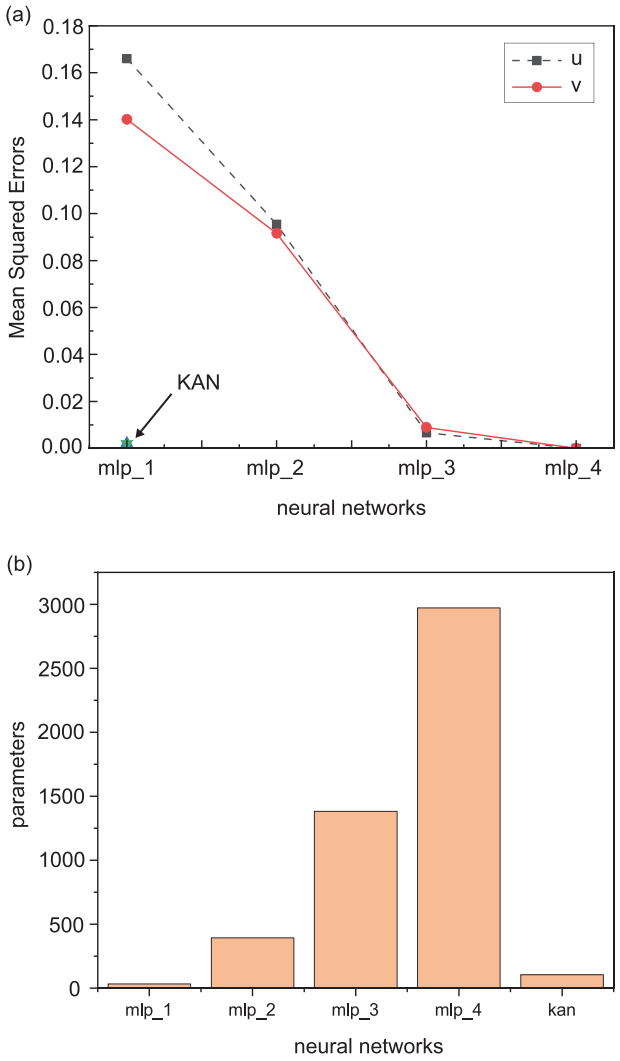

FIG. 11. Mean squared error curves for velocity predictions obtained from different neural network models, along with bar charts showing the parameter counts for each network. (a) The evolution of the mean squared error (MSE) for velocity predictions obtained from training four MLP models, along with a comparison to the KAN model. (b) A bar chart comparing the total parameter counts across five neural network models.

MLP models, as seen with mlp_2 and mlp_3, prediction accuracy improves significantly. However, even mlp_3 still falls short of the baseline accuracy achieved by PI-KANs. When the number of neurons is further increased in mlp_4, the prediction accuracy surpasses that of PI-KANs, but with a model that has 30 times more parameters. Figure 11(b) shows a bar chart of the parameter counts, clearly illustrating the difference in the number of parameters between mlp_4 and PI-KANs.

For the problem studied in this paper, both PI-KANs and MLPs can achieve satisfactory accuracy, but each has its own characteristics. Using MLPs requires constructing a relatively large model with deeper layers and more parameters, a smaller learning rate, and typically using both Adam and LBFGS optimizers. Each iteration step takes less time, but more iterations are needed, generally using ReLU and tanh activation functions. Additionally, MLPs lack clear interpretability, making it difficult to gain insights into the physical mechanisms of the problem. On the other hand, PI-KANs typically use a smaller network with fewer parameters, require longer iteration times, but generally converge faster to high accuracy with fewer training steps. Moreover, the use of B-splines as activation functions allows for better visualizations, providing deeper insights into the inner workings of the network, which can improve the physical understanding of the problem. For certain scientific problems, this interpretability may be more advantageous than symbolic regression methods. Future research will focus on exploring this aspect further.

IV. CONCLUSION

In this study, we developed a Physics-Informed Kolmogorov–Arnold Networks (PI-KANs) to solve the two-dimensional Taylor–Green vortex problem and evaluated its predictive performance.

First, we investigated the effect of random seed initialization on the prediction accuracy of a large KAN architecture. Using a network configuration of [2,5,5,5,5,3], we tested eight random seeds and analyzed the predicted velocity fields under each condition. The results revealed that random seeds significantly influence both the prediction accuracy and training speed of the network. Additionally, we observed that the pde loss constituted a substantial proportion of the total loss during training. After identifying a network with high prediction accuracy using the large KANs and appropriate random seed, we applied pruning techniques to reduce the network size while maintaining accuracy. However, after progressively increasing the pruning threshold, we found that for multi-output networks, achieving a compact network through pruning is not straightforward. Evaluations of MSE on trained collocation points and test collocation points showed that, provided the number of trained collocation points was not too small, the MSE calculated on trained points could reliably reflect the MSE on test points.

Second, we explored the information bottleneck issue in the two-dimensional Taylor–Green vortex problem. By training networks with varying architectures, we observed that networks with insufficient neurons in the hidden layers could not achieve satisfactory predictions, even with adjustments to other hyperparameters. Adding a single neuron to the hidden layer substantially improved prediction accuracy, but further increases in neuron count did not significantly enhance the performance. Additionally, we studied the effect of the weighting factor α on training, which controls the proportion of pde loss in the total loss. We found that increasing α led to poor prediction performance and hindered network convergence.

Third, we examined the impact of collocation point quantity on network performance. Training the network with different numbers of collocation points revealed that using too few points resulted in poor predictions. As the number of collocation points increased beyond a certain threshold, the network achieved high prediction accuracy. This threshold is likely problem-specific. In the present study, excessive collocation points were not necessary to achieve satisfactory accuracy.

Then, we investigated the influence of network shape (U-shape, N-shape, and Uniform shape) on prediction performance under equal parameter constraints. The results showed that the Uniform shape provided the best performance, followed by the N-shape, while the U-shape performed the worst. These findings suggest that selecting an appropriate network shape requires prior analysis and consideration of the specific problem at hand.

Finally, we conducted a comparative study between PI-KANs and MLP-based PINNs for solving the Navier-Stokes equations, highlighting the respective advantages and disadvantages of both neural networks. As for their performance in other scientific fields, further exploration is needed to assess the capabilities of these two neural networks.

This study highlights the critical roles of random seed initialization, collocation point distribution, network architecture, and hyperparameter tuning in the performance of PI-KANs. These insights can serve as a reference for optimizing PI-KANs for other fluid dynamics problems and beyond.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Shuangwei Cui: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Project administration (equal); Resources (equal); Software (equal); Supervision (equal); Validation (equal); Visualization (equal); Writing – original draft (equal). **Manshu Cao:** Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Resources (equal); Validation (equal); Visualization (equal). **Yifeng Liao:** Data curation (equal); Formal analysis (equal); Investigation (equal); Resources (equal); Validation (equal); Visualization (equal). **Jianing Wu:** Conceptualization (equal); Funding acquisition (equal); Investigation (equal); Supervision (equal); Validation (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

REFERENCES

[1]Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE **86**, 2278–2324 (1998).
[2]L. R. Medsker and L. Jain *et al.*, "Recurrent neural networks," Des. Appl. **5**, 64–67, 2 (2001).
[3]Y.-G. Ham, J.-H. Kim, and J.-J. Luo, "Deep learning for multi-year ENSO forecasts," Nature **573**, 568–572 (2019).
[4]I. P. Waldmann and C. A. Griffith, "Mapping saturn using deep learning," Nat. Astron. **3**, 620–625 (2019).

[5]J. N. Kutz, "Deep learning in fluid dynamics," J. Fluid Mech. **814**, 1–4 (2017).
[6]K. Duraisamy, G. Iaccarino, and H. Xiao, "Turbulence modeling in the age of data," Annu. Rev. Fluid Mech. **51**, 357–377 (2019).
[7]S. L. Brunton, B. R. Noack, and P. Koumoutsakos, "Machine learning for fluid mechanics," Annu. Rev. Fluid Mech. **52**, 477–508 (2020).
[8]H. Eivazi, M. Tahani, P. Schlatter, and R. Vinuesa, "Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations," Phys. Fluids **34**, 075117 (2022).
[9]Y. Zhu, Y. Zhu, Z. Ding, H. Ding, R. Zhou, Y. Liao, and J. Wu, "Rapid flow field prediction in patterned baleen membranes of balaenid whales during filter feeding by deep learning," Phys. Fluids **36**, 081912 (2024).
[10]R. Temam, *Navier–Stokes Equations: Theory and Numerical Analysis* (American Mathematical Society, 2024), Vol. 343.
[11]J. C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations* (SIAM, 2004).
[12]R. Gallagher, J. Oden, C. Taylor, and O. Zienkiewicz, *Finite Element Analysis: Fundamentals* (Prentice Hall, Englewood Cliffs, NJ, 1975).
[13]T. A. Zang, C. L. Streett, and M. Y. Hussaini, "Spectral methods for CFD," Technical report (1989).
[14]M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys. **378**, 686–707 (2019).
[15]X. Jin, S. Cai, H. Li, and G. E. Karniadakis, "NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations," J. Comput. Phys. **426**, 109951 (2021).
[16]S. Cai, Z. Wang, F. Fuest, Y. J. Jeon, C. Gray, and G. E. Karniadakis, "Flow over an espresso cup: Inferring 3-d velocity and pressure fields from tomographic background oriented schlieren via physics-informed neural networks," J. Fluid Mech. **915**, A102 (2021).
[17]H. Eivazi, Y. Wang, and R. Vinuesa, "Physics-informed deep-learning applications to experimental fluid mechanics," Meas. Sci. Technol. **35**, 075303 (2024).
[18]G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," Nat. Rev. Phys. **3**, 422–440 (2021).
[19]Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, KAN: Kolmogorov-Arnold networks," arXiv:2404.19756 (2024).
[20]A. Kolmogorov, *On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables*.
[21]A. N. Kolmogorov, "On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition," Doklady Akademii Nauk **114**, 953–956 (1957).
[22]J. Braun and M. Griebel, "On a constructive proof of Kolmogorov's superposition theorem," Constr. Approx. **30**, 653–675 (2009).
[23]S. Haykin, *Neural Networks: A Comprehensive Foundation* (Prentice Hall PTR, 1998).
[24]G. Cybenko, "Approximation by superpositions of a sigmoidal function," Math. Control. Signal. Syst. **2**, 303–314 (1989).
[25]K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," Neural Netw. **2**, 359–366 (1989).
[26]T. Hastie *et al.*, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (Springer, 2009).
[27]J. Han and C. Moraga, "The influence of the sigmoid function parameters on the speed of backpropagation learning," in *International Workshop on Artificial Neural Networks* (Springer, 1995), pp. 195–201.
[28]J. Brownlee, "A gentle introduction to the rectified linear unit (relu)," Mach. Learn. Mastery **6** (2019), available at https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/.
[29] See https://mathworld.wolfram.com/ for more information about hyperbolic functions.
[30]D. A. Forsyth and J. Ponce, *Computer Vision: A Modern Approach* (Prentice Hall Professional Technical Reference, 2002).
[31]R. Mihalcea, H. Liu, and H. Lieberman, "Nlp (natural language processing) for nlp (natural language programming)," in *Computational Linguistics and Intelligent Text Processing: 7th International Conference, CICLing 2006, Mexico City, Mexico, February 19–25, 2006* (Springer, 2006), pp. 319–330.
[32]L. Bottou and O. Bousquet, "The tradeoffs of large scale learning," in Proceedings of the 21st International Conference on Neural Information Processing Systems (2007), pp. 161–168.
[33]A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," J. Mach. Learn. Res. **18**, 1–43 (2018), available at http://jmlr.org/papers/v18/17-468.html.
[34]C. Foias, O. Manley, R. Rosa, and R. Temam, *Navier-Stokes Equations and Turbulence* (Cambridge University Press, 2001), Vol. 83.
[35]C. Wang, "Exact solutions of the unsteady Navier-Stokes equations," Appl. Mech. Rev. **42**, S269 (1989).
[36]C. R. Ethier and D. Steinman, "Exact fully 3D Navier–Stokes solutions for benchmarking," Int. J. Numer. Methods Fluids **19**, 369–375 (1994).
[37]M. E. Brachet, D. I. Meiron, S. A. Orszag, B. Nickel, R. H. Morf, and U. Frisch, "Small-scale structure of the Taylor–Green vortex," J. Fluid Mech. **130**, 411–452 (1983).
[38]G. Łukaszewicz and P. Kalita, *Navier–Stokes Equations: An Introduction with Applications* (Springer, 2016), Vol. 34.
[39]A. Green and G. Taylor, "Mechanism of the production of small eddies from larger ones," Proc. R. Soc. A **158**, 499–521 (1937).
[40]D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," Math. Program. **45**, 503–528 (1989).